

Appunti Tirocinio

Alessio Marini

Appunti presi per il mio Tirocinio interno presso il Vision-Lab.

July 19, 2026

1. Linear Algebra

1.1. Scalars

I numeri che utilizziamo tutti i giorni prendono il nome di **scalari**, li indicheremo con lettere *lower-case* come x, y, z e lo spazio di tutti gli scalari **reali** $\mathbb{R}$.

Implementiamo gli scalari come dei tensor da un solo elemento, vediamo un esempio di assegnamento e delle operazioni:

python

```
x = torch.tensor(3.0)
y = torch.tensor(2.0)

x + y, x * y, x / y, x**y
# Output
# (tensor(5.), tensor(6.), tensor(1.5000), tensor(9.))
```

1.2. Vectors

Possiamo vedere i vettori come degli array di scalari con lunghezza fissa, gli scalari saranno gli *elementi* del vettore. Quando li utilizzeremo per rappresentare problemi reali avremo che ogni elemento indica un parametro di cosa stiamo analizzando. Indichiamo i vettore con lettere lowercase bold come $\mathbf{x}, \mathbf{y}, \mathbf{z}$.

Ci riferiamo agli elementi di un vettore con la lettera del vettore e l'indice dell'elemento in basse, x_2 indica l'elemento con indice 2 nel vettore $\mathbf{x}$. Normalmente li visualizziamo impilando gli elementi verticalmente:

$$\mathbf{x} = \begin{bmatrix} x_1 \\ \vdots \\ x_3 \end{bmatrix}$$

Per indicare che un vettore contiene n elementi scriviamo che $\mathbf{x} \in \mathbb{R}^n$, formalmente diciamo che n è la **dimensione** dell'array, questa nel codice corrisponde appunto alla lunghezza del tensor che possiamo richiamare con `len(x)` oppure tramite l'attributo `x.shape`.

Per fare chiarezza indichiamo con:

- **Order**: Numero degli assi di un vettore
- **Dimensionality**: Numeri dei componenti

1.3. Matrici

Gli scalari sono tensor di ordine 0, i vettori sono tensor di ordine 1 e le matrici sono di ordine 2. Le indichiamo con lettere upper case bold, quindi $\mathbf{X}, \mathbf{Y}, \mathbf{Z}$ e le rappresentiamo nel codice con vettori con due assi. Con l'espressione $\mathbf{A} \in \mathbb{R}^{m \times n}$ indichiamo che una matrice $\mathbf{A}$ contiene $m \times n$ valori reali ordinati in m righe e n colonne. Quando $m = n$ diciamo che la matrice è *quadrata*. Possiamo illustrarla come una tabella e per indicare un elemento usiamo il doppio pedice, quindi $a_{i,j}$ indica l'elemento nella i -esima riga e alla j -esima colonna di $\mathbf{A}$.

$$\mathbf{A} = \begin{bmatrix} a_{11} & a_{12} & \dots & a_{1n} \\ a_{21} & a_{22} & \dots & a_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ a_{m1} & a_{m2} & \dots & a_{mn} \end{bmatrix}$$

É possibile invertire gli assi ovvero scambiare righe con colonne, il risultato é la **matrice trasposta**, la indichiamo con $\mathbf{A}^\top$. Otteniamo che se $\mathbf{B} = \mathbf{A}^\top$ allora $b_{ij} = a_{ji}$ per ogni i, j . Inoltre la trasposta di una matrice $m \times n$ sar  una matrice $n \times m$.

Le matrici **simmetriche** sono un sottoinsieme delle matrici quadrate, sono matrici uguali alla loro trasposta, quindi $\mathbf{A} = \mathbf{A}^\top$

1.4. Tensors

I tensors ci danno modo di rappresentare array di ordine n . Li indichiamo con lettere maiuscole X,Y,Z e per gli elementi utilizziamo lo stesso meccanismo di indici delle matrici. I tensors diventeranno pi  importanti quando lavoreremo con le immagini, ognuna infatti é rappresentata da un tensor di ordine 3 dove gli assi corrispondono a *altezza*, *larghezza* e *canali*. Una collezione di immagini sar  quindi un tensor di ordine 4.

1.4.1. Basic Properties of Tensor Arithmetic

Scalari, vettori, matrici e tensors hanno delle propriet , ad esempio le operazioni *element-wise* producono output della stessa forma degli operandi.

python

```
A = torch.arange(6, dtype=torch.float32).reshape(2, 3)
B = A.clone() # Alloca nuova memoria per B
A, A + B

# Output
# (tensor([[0., 1., 2.],
#          [3., 4., 5.]]),
#  tensor([[ 0.,  2.,  4.],
#          [ 6.,  8., 10.]])
```

Il prodotto *elementwise* di due matrici si chiama **Hadamard Product**, lo indichiamo con $\odot$. Gli elementi saranno quindi:

$$\mathbf{A} \odot \mathbf{B} = \begin{bmatrix} a_{11}b_{11} & a_{12}b_{12} & \dots & a_{1n}b_{1n} \\ a_{21}b_{21} & a_{22}b_{22} & \dots & a_{2n}b_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ a_{m1}b_{m1} & a_{m2}b_{m2} & \dots & a_{mn}b_{mn} \end{bmatrix}$$

Andando invece a sommare o moltiplicare un tensor con uno scalare otteniamo un tensor con la stessa forma di quello originale ma su ogni elemento viene applicata l'operazione specificata.

1.4.2. Reduction

Spesso ci capiter  di dover calcolare la somma di tutti gli elementi di un tensor. Matematicamente possiamo esprimerla come $\sum_{i=1}^n x_i$ ma nel codice esiste una funzione c'python

```
x = torch.arange(3, dtype=torch.float32)
x, x.sum()
```

```
# Output
# (tensor([0., 1., 2.]), tensor(3.))
```

La funzione `sum()` va bene anche per tensors di ordine maggiore. Di base chiamare la funzione somma, *riduce* il tensor su tutti i suoi assi producendo, in alcuni casi, uno scalare. Possiamo specificare su quali assi effettuare la somma e quell'asse ovviamente sparirà dalla dimensione del tensor:

python

```
A.shape, A.sum(axis=0).shape
# Output
# (torch.Size([2, 3]), torch.Size([3]))

A.shape, A.sum(axis=1).shape
# Output
# (torch.Size([2, 3]), torch.Size([2]))
```

In alcuni casi però potrebbe essere utile si calcolare la somma, ma anche mantenere intatto il tensor, in questi casi impostiamo:

python

```
sum_A = A.sum(axis=1, keepdims=True)
sum_A, sum_A.shape

# Output
# (tensor([[ 3.],
#          [12.]]),
# torch.Size([2, 1]))
```

1.4.3. Dot Products

È una delle operazioni principali, dati due vettori $\mathbf{x}, \mathbf{y} \in \mathbb{R}^d$ il loro *dot product* è dato da $\mathbf{x}^\top \mathbf{y}$, ovvero la somma dei prodotti degli elementi nella stessa posizione:

$$\mathbf{x}^\top \mathbf{y} = \sum_{i=1}^d x_i y_i$$

python

```
y = torch.ones(3, dtype = torch.float32)
x, y, torch.dot(x, y)

# Output
# (tensor([0., 1., 2.]), tensor([1., 1., 1.]), tensor(3.))
```

In modo perfettamente equivalente possiamo calcolare un dot product effettuando prima un prodotto elementwise e poi una somma.

I dot product sono fondamentali in questo ambito. Ad esempio dato un set di valori in un vettore $\mathbf{x} \in \mathbb{R}^n$ e un set di pesi $\mathbf{w} \in \mathbb{R}^n$; la *weighted sum* di $\mathbf{x}$ in base ai pesi $\mathbf{w}$ può essere espressa come il dot product $\mathbf{x}^\top \mathbf{w}$.

Quando i pesi sono non negati e la loro somma è uguale a 1, il dot product rappresenta una *weighted average*. Inoltre, dopo aver normalizzato due vettori in modo da avere una lunghezza unitaria (pari a 1) il dot product esprimerà il coseno dell'angolo compreso fra loro.

1.4.4. Matrix - Vector Products

Adesso possiamo vedere il prodotto fra una matrice $m \times n$ $\mathbf{A}$ e un vettore n -dimensionale $\mathbf{x}$. Per prima cosa visualizziamo la matrice come vettori riga:

$$\mathbf{A} = \begin{bmatrix} \mathbf{a}_1^\top \\ \mathbf{a}_2^\top \\ \vdots \\ \mathbf{a}_m^\top \end{bmatrix}$$

Dove ogni $\mathbf{a}_i^\top \in \mathbb{R}^n$ é un vettore-riga che rappresenta la i -esima riga di $\mathbf{A}$.

Il prodotto matrice-vettore $\mathbf{Ax}$ é semplicemente un vettore colonna di lunghezza m dove l'elemento i é il dot product $\mathbf{a}_i^\top \mathbf{x}$:

$$\mathbf{Ax} = \begin{bmatrix} \mathbf{a}_1^\top \\ \mathbf{a}_2^\top \\ \vdots \\ \mathbf{a}_m^\top \end{bmatrix} \mathbf{x} = \begin{bmatrix} \mathbf{a}_1^\top \mathbf{x} \\ \mathbf{a}_2^\top \mathbf{x} \\ \vdots \\ \mathbf{a}_m^\top \mathbf{x} \end{bmatrix}$$

Possiamo vedere quindi la moltiplicazione con una matrice $\mathbf{A} \in \mathbb{R}^{m \times n}$ come una trasformazione che porta i vettori da $\mathbb{R}^n$ a $\mathbb{R}^m$. Queste trasformazioni sono molto utili infatti sono l'operazione principale usata in ogni layer dove, dato l'output del layer precedente, viene calcolato l'output attuale.

Per rappresentare questi prodotti nel codice usiamo la funzione `mv`. Per effettuarla é importante che la dimensione di $\mathbf{A}$ sia la stessa di $\mathbf{x}$. In Python inoltre abbiamo un operatore pensato sia per prodotti matrix-matrix che matrix-vector che é `@`:

python

```
A.shape, x.shape, torch.mv(A, x), A@x
```

```
# Output
```

```
# (torch.Size([2, 3]), torch.Size([3]), tensor([ 5., 14.]), tensor([ 5., 14.]))
```

1.4.5. Matrix - Matrix Multiplication

Il concetto é molto simile a quello dei dot product. Prendiamo due matrici $\mathbf{A} \in \mathbb{R}^{n \times k}$ e $\mathbf{B} \in \mathbb{R}^{k \times m}$:

$$\mathbf{A} = \begin{bmatrix} a_{11} & a_{12} & \dots & a_{1k} \\ a_{21} & a_{22} & \dots & a_{2k} \\ \vdots & \vdots & \ddots & \vdots \\ a_{n1} & a_{n2} & \dots & a_{nk} \end{bmatrix}, \mathbf{B} = \begin{bmatrix} b_{11} & b_{12} & \dots & b_{1m} \\ b_{21} & b_{22} & \dots & b_{2m} \\ \vdots & \vdots & \ddots & \vdots \\ a_{k1} & a_{k2} & \dots & a_{km} \end{bmatrix}$$

Sia $\mathbf{a}_i^\top \in \mathbb{R}^k$ il vettore-riga che rappresenta la i -esima riga della matrice $\mathbf{A}$ e sia $\mathbf{b}_j \in \mathbb{R}^k$ il vettore-colonna che rappresenta la j -esima colonna della matrice $\mathbf{B}$:

$$\mathbf{A} = \begin{bmatrix} \mathbf{a}_1^\top \\ \mathbf{a}_2^\top \\ \vdots \\ \mathbf{a}_n^\top \end{bmatrix}, \mathbf{B} = [\mathbf{b}_1 \ \mathbf{b}_2 \ \dots \ \mathbf{b}_m]$$

Per formare la matrice $\mathbf{C} \in \mathbb{R}^{n \times m}$ calcoliamo ogni elemento c_{ij} come il dot product tra la i -esima riga di $\mathbf{A}$ e la j -esima colonna di $\mathbf{B}$:

$$C = AB = \begin{bmatrix} \mathbf{a}_1^\top \\ \mathbf{a}_2^\top \\ \vdots \\ \mathbf{a}_n^\top \end{bmatrix} [\mathbf{b}_1 \ \mathbf{b}_2 \ \dots \ \mathbf{b}_m] = \begin{bmatrix} \mathbf{a}_1^\top \mathbf{b}_1 & \mathbf{a}_1^\top \mathbf{b}_2 & \dots & \mathbf{a}_1^\top \mathbf{b}_m \\ \mathbf{a}_2^\top \mathbf{b}_1 & \mathbf{a}_2^\top \mathbf{b}_2 & \dots & \mathbf{a}_2^\top \mathbf{b}_m \\ \vdots & \vdots & \ddots & \vdots \\ \mathbf{a}_n^\top \mathbf{b}_1 & \mathbf{a}_n^\top \mathbf{b}_2 & \dots & \mathbf{a}_n^\top \mathbf{b}_m \end{bmatrix}$$

Possiamo pensare ad una moltiplicazione matrix-matrix $\mathbf{AB}$ equivalente ad effettuare m prodotti matrix-vector o $m \times n$ dot products per mettere insieme i risultati e ottenere una matrice $n \times m$.